



BANK HOLIDAY & SERVICECALENDAR

PROJECTDOCUMENTATION

A Centralized Portal for Holiday, Downtime & Branch HoursManagement

Subject: Front End Development Frameworks (FEDF)

Technology Stack: React.js · React Router v6 · LocalStorage · CSS3 · Vite

Total Modules: 10 Functional Modules · Version: Final

BATCH NO: 20

G. Hansika Reddy (Roll No: 2520030237)

K. Anusri (Roll No: 2520030432)

Under the Esteemed Guidance of

Dr. Lakshmi Prasanna

Department of Front End Development Frameworks and UI Engineering

SECTION —

Acknowledgement

The success of this project would not have been possible without the timely help and guidance rendered by many individuals. We wish to express our sincere gratitude to all those who assisted us in one way or another for the successful completion of this project.

Our deepest appreciation goes to our guide, **Dr. Lakshmi Prasanna**, Department of Front End Development Frameworks and UI Engineering, whose tremendous support, encouragement, and expert guidance throughout this project cannot be expressed in words. Her insights into modern frontend development practices and her consistent mentorship have been the cornerstone of this project's success.

We express our gratitude to the Department of Front End Development Frameworks and UI Engineering for providing us with adequate facilities, technical resources, and the necessary infrastructure that enabled us to design, develop, and deploy this full-featured React.js application.

We thank all the members of the teaching and non-teaching staff who have assisted us directly or indirectly in the successful completion of this project. Their encouragement during lab sessions and code reviews helped us identify and resolve critical implementation challenges across all ten modules.

Finally, we sincerely thank our parents, friends, and classmates for their unwavering support, motivation, and cooperation during the course of this work. Special thanks to our batchmates for their valuable feedback during testing and peer review sessions.

G. Hansika Reddy

Roll No: 2520030237

Batch No: 20

K. Anusri

Roll No: 2520030432

Batch No: 20

SECTION —

Table of Contents

Section	Title	Page
1	Project Title & Overview	4
2	Purpose & Objectives	5
3	Problem Statement	6
4	Project Goals	7
4.1	Primary Goals	7
4.2	Functional Goals	7
4.3	FEDF Learning Goals	8
5	Scope of the Project	9
5.1	In Scope Features	9
5.2	Out of Scope	9
6	User Personas & Roles	10
7	System Architecture	11
8	Technology Stack	12
9	Project File Structure	13
10	Functional Requirements	14
10.1	Authentication Module	14
10.2	Holiday Management	15
10.3	Service Downtime	15
10.4	Branch Hours	16
10.5	Notification Centre	16
10.6	Audit History	17
10.7	Admin Panel	17
10.8	Export Module	17

11	Non-Functional Requirements	18
12	Module: Home & Authentication	19
13	Module: Holiday List & Add Holiday	20
14	Module: Service Downtime Scheduler	21
15	Module: Branch Hours Management	22
16	Module: Notification Centre	23
17	Module: Audit History	24
18	Module: Admin Panel	25
19	Module: Export Module	26
20	Version Roadmap	27
21	Acceptance Criteria	28
22	Assumptions & Constraints	29
23	Conclusion	30

SECTION 1

Project Title & Overview

Project Title: Bank Holiday & Service Calendar — A Centralized Portal for Holiday, Downtime & Branch Hours Management

Subject: Front End Development Frameworks (FEDF)

Technology: React.js 18 · React Router v6 · LocalStorage · CSS3 · Vite · JavaScript ES6+

Modules: 10 Functional Modules implemented across V1, V2, and Final Version

Academic Year: 2026 · **Institution:** Department of FEDF and UI Engineering

1.1 Project Introduction

The Bank Holiday & Service Calendar is a comprehensive, production-ready React.js web application developed as the Final FEDF project submission. The portal serves as a unified platform enabling bank operations teams to publish, manage, and communicate holiday schedules, service downtime windows, and branch operational hours in real time.

India's banking landscape consists of nationalized banks, private banks, cooperative banks, and regional rural banks — each governed by different sets of holiday policies issued by the Reserve Bank of India (RBI), respective state governments, and internal bank calendars. This complexity creates a significant information gap for customers and businesses who rely on banking services for their day-to-day financial transactions.

1.2 Key Highlights

Feature	Description
10 Functional Modules	Authentication, Holiday Management, Add Holiday, Service Downtime, Branch Hours, Notification Centre, Audit History, Admin Panel, Export Module, and the Home dashboard.
Role-Based Access Control	Two distinct user roles — Admin (full access) and User (read-only) — with route guards protecting all sensitive operations.
200+ Pre-Seeded Holidays	Complete holiday data across 26 Indian states and Union Territories for the 2026 calendar year.
Three Export Formats	Download holiday data as CSV spreadsheet, iCal (.ics) for calendar apps, or print-to-PDF via browser.
Real-Time Audit Logging	Every significant portal action is timestamped and logged with the acting user's identity.

SECTION 2

Purpose & Objectives

The core purpose of this portal is to eliminate the confusion caused by scattered, inconsistent communication of bank holidays and service disruptions across different channels. By providing a unified, role-protected platform, the portal empowers bank operations staff to update schedules in real time and proactively notify customers via SMS and email alerts — all from a single, intuitive dashboard.

2.1 Primary Objectives

- Provide a **centralized, searchable directory** of all bank holidays across Indian states and regions.
- Allow **administrators** to manage holiday data, branch hours, and service downtime from a secure dashboard.
- Enable **real-time customer notifications** via SMS and email channels with customizable templates.
- Maintain a **complete, tamper-evident audit trail** of all changes made across the portal.
- Allow **export of holiday data** in CSV, iCal (.ics), and print-to-PDF formats.
- Demonstrate mastery of **React.js component architecture**, routing, and state management.
- Implement **localStorage as a lightweight data layer** for a purely client-side application.

2.2 Business Value

For banking organizations, this portal reduces operational overhead by consolidating holiday communications into a single system. Customer-facing staff can respond authoritatively to holiday-related queries. Operations managers can schedule and track planned maintenance windows with full lifecycle management. Compliance teams gain complete audit visibility for every change made to the system — satisfying internal governance and audit requirements.

2.3 Academic Value

This project demonstrates advanced proficiency in front-end development frameworks — specifically React.js 18 with functional components and hooks. It covers the complete FEDF curriculum including declarative rendering, controlled forms, client-side routing, protected routes, localStorage persistence, async UI patterns, responsive CSS, and performance optimization via Vite.

SECTION 3

Problem Statement

In India, bank holidays vary significantly across states, banks, and regions — governed by RBI guidelines, state government declarations, and individual bank policies. Despite this complexity, there is currently no single, reliable platform where customers or businesses can verify whether a specific branch is open or a banking service is available on a given day.

Problem Area	Impact
Customer Frustration	Customers frequently visit branches on closed days — wasting time, travel expenses, and creating frustration due to lack of prior notification.
Payment Failures	Payments and transfers fail during unannounced maintenance windows, causing financial disruption for businesses and individuals relying on NEFT/RTGS/UPI services.
Deadline Misses	Businesses miss critical financial deadlines — GST payments, loan EMIs, salary disbursements — by acting on outdated or incorrect holiday information.
Fragmented Communication	Different banks communicate holidays through different channels: some via SMS, some via mobile app, some via website notices, and many not at all.
No Unified Admin System	Bank staff have no unified system to manage, update, or broadcast holiday and downtime information — forcing them to use email chains and manual notices.
Zero Audit Visibility	Changes to branch hours, holidays, or service availability are made ad-hoc with no audit trail, creating compliance and governance risks.

3.1 The Solution

The NationalBank Portal addresses all of the above by consolidating holiday and service information into one secure, role-based web application. Admins can add/manage holidays, schedule maintenance, edit branch hours, send notifications, view audit logs, and export data — while customers and staff can access current, accurate information at any time.

SECTION 4

Project Goals

4.1 Primary Goals

- Build a **production-ready React.js web application** for managing bank holidays and service schedules.
- Implement **role-based access control** (Admin / User) with localStorage-based authentication.
- Deliver a **responsive UI** that works correctly on both desktop and mobile devices.
- Implement all **10 modules** required by the FEDF project specification.
- Ensure **full data persistence** via browser localStorage across all modules.
- Apply **React best practices**: functional components, hooks, controlled forms, and unidirectional data flow.

4.2 Functional Goals

Module	Functional Goal
Holiday Management	Display 200+ holidays across 26 Indian states with dual list / calendar view
Add Holiday	Admin-only form with field validation, duplicate check, and audit logging
Service Downtime	Schedule maintenance windows with full lifecycle status management (Scheduled → Active → Completed)
Branch Hours	City-wise operational hours with admin inline-edit capability per branch
Notification Centre	Compose SMS/email alerts with three built-in message templates and draft workflow
Audit History	Timestamped log of 9 action types, filterable by action type, user, and keyword search
Admin Panel	User management — view, role assignment, and account deletion with self-protection
Export Module	CSV download, iCal (.ics) file generation, and print-to-PDF with state and type filters

4.3 FEDF Learning Goals

Module	Learning Objectives
Module 1	Declarative rendering, component composition, unidirectional data flow
Module 2	Async data handling, typed exports, action history management
Module 3	Controlled forms, props-driven components, hook-based state (useState, useEffect)
Module 4	LocalStorage as data layer, async UI patterns, optimistic UI updates
Module 5	React Router v6, protected routes, breadcrumb navigation, ARIA accessibility
Module 6	Vite build system, CI/CD pipeline, Lighthouse performance audits

SECTION 5

Scope of the Project

5.1 In Scope

Feature Area	What Is Included
Authentication	Registration, login/logout, role selection (Admin/User), navigation guards for all protected routes
Holiday Management	Display, search, filter, add, and delete holidays — 200+ entries covering all 26 states and 5 UTs
Service Downtime	Schedule and manage lifecycle (Scheduled → Active → Completed) for 10 banking systems
Branch Hours	View and edit operational hours for bank branches across 8+ major Indian cities
Notification Centre	Compose and log SMS/email alerts with 3 templates; draft and send workflows
Audit History	Full activity log with filters for 9 action types, user, and keyword search
Admin Panel	View all users, change roles, delete accounts; self-deletion blocked
Export Module	CSV download, iCal (.ics) file, print-to-PDF with state and holiday type filters
UI/UX	Responsive design, consistent header/footer, breadcrumb navigation, role-based access-denied states

5.2 Out of Scope

- Real backend API or database integration — project uses localStorage for all data persistence
- Actual SMS or email delivery — notifications are simulated and logged locally only
- Multi-bank or cross-institution data federation
- Payment gateway, transaction processing, or core banking functionality
- Mobile native applications for iOS or Android
- Integration with RBI or government holiday data APIs
- Multi-language (i18n) or regional language support
- Two-factor authentication or OAuth integration

SECTION 6

User Personas & Roles

The portal serves two distinct user roles, each with a clearly defined permission set. Role assignment occurs at registration and can be changed by an administrator through the Admin Panel.

Role	Who They Are	Access Level	Key Permissions
Admin	Bank operations staff, IT managers, branch administrators	Full Access	Add/delete holidays, manage downtime, send notifications, edit branch hours, manage users, view audit log, export data
User (Viewer)	Bank customers, employees, general public	Read-Only	View holiday list, calendar, branch hours, downtime schedules, notification history

6.1 User Persona: Admin — Ravi Kumar

Role: Bank Operations Manager, Hyderabad Branch Network

Goal: Ensure all branch staff and customers have accurate, up-to-date holiday and service information.

Typical Workflow: Ravi logs in to the Admin portal before major festivals to add state-specific holidays for Telangana and Andhra Pradesh. He schedules ATM maintenance windows for the upcoming weekend, ensuring downtime is set to 'Scheduled' status. Before the Ganesh Chaturthi long weekend, he uses the Notification Centre to compose a branch closure alert using the Holiday Closure template and sends it to all customers. He exports the October holiday list as a CSV for distribution to the regional team.

6.2 User Persona: User — Priya Sharma

Role: Retail Banking Customer, Hyderabad

Goal: Verify branch availability and NEFT service status before planning financial transactions.

Typical Workflow: Priya needs to transfer money for a property deal. She visits the portal, filters the Holiday List by Telangana, and confirms the transaction date is not a state holiday. She checks the Service Downtime page to ensure NEFT/RTGS services are available. She also downloads the iCal file to sync bank holidays with her Google Calendar for the entire year.

SECTION 7

System Architecture

7.1 Application Architecture

The NationalBank Portal follows a component-based Single Page Application (SPA) architecture using React.js 18. The application uses client-side routing via React Router v6 and browser localStorage as its data persistence layer. The architecture is deliberately stateless at the server level, making the application deployable as a static site.

Layer	Technology	Responsibility
Presentation	React.js 18 + CSS3	Component rendering, user interaction, responsive layout
Routing	React Router v6	Client-side navigation, HashRouter, route protection
State Management	React useState / useEffect	Local component state, side effects, data loading
Data Persistence	Browser LocalStorage	Users, holidays, audit log, notifications, branch hours
Build System	Vite	Fast HMR development server, production bundling, asset optimization
Version Control	Git / GitHub	Source control, collaboration, deployment pipeline

7.2 Data Flow

The application follows a unidirectional data flow pattern. User interactions trigger event handlers which update React state. State changes cause component re-renders. Persistent data changes are simultaneously written to localStorage using helper functions (e.g., saveDowntimes(), saveNotifications()). On page load, components read from localStorage using seed functions that populate default data if none exists (getDowntimes(), getAuditLog(), etc.).

7.3 LocalStorage Data Schema

Key	Type	Contents
users	Array	Registered users: {username, password, role}
isLoggedIn	String	'true' when a session is active
username	String	Current logged-in user's username
holidays_v2	Array	All holiday entries with full metadata
holidays	Array	Legacy key maintained for export compatibility
downtimes	Array	All service downtime entries
branchHours	Object	Branch-wise operational hours keyed by branch name
notifications	Array	All sent/drafted notification entries
auditLog	Array	All audit entries: {action, detail, user, at}

SECTION 8

Technology Stack

The NationalBank Portal is built entirely using modern JavaScript and React ecosystem tools. The technology choices prioritize developer productivity, bundle performance, and browser compatibility. No backend framework, database, or server infrastructure is required.

Category	Technology	Version	Purpose
UI Framework	React.js	18.x	Component-based declarative UI development with hooks
Routing	React Router	v6	Client-side navigation, HashRouter for static hosting compatibility
Build Tool	Vite	5.x	Ultra-fast HMR dev server, ES module bundling, static output
Styling	Custom CSS3	—	Responsive layout with CSS Grid, Flexbox, and CSS variables
State Persistence	localStorage API	Browser	Users, holidays, audit log, notifications, settings
Language	JavaScript	ES2022+	Async/await, destructuring, optional chaining, arrow functions
Package Manager	npm	9+	Dependency management and script running
Version Control	Git / GitHub	—	Source control, collaboration, and static deployment
Deployment	Netlify / GitHub Pages	—	Static site hosting with automatic deployment on push

8.1 React Features Used

- **useState** — all component-level state management (forms, filters, modal toggles, lists)
- **useEffect** — data loading from localStorage on component mount and storage event listeners
- **useNavigate** — programmatic navigation after form submissions and authentication checks
- **Link / NavLink** — declarative navigation with breadcrumb trail support
- **HashRouter** — client-side routing compatible with static file hosting (Netlify, GitHub Pages)
- **Controlled Components** — all form inputs managed via state with onChange handlers
- **Conditional Rendering** — role-based UI (admin vs user view), access-denied screens, empty states
- **Component Composition** — shared Header, Footer, and reusable card components across all pages

SECTION 9

Project File Structure

The project follows the standard Vite + React project structure, with source files organized by type — pages for top-level route components and components for feature-specific modules. All styling is centralized in a single CSS file for consistency.

	Description
FINAL FEDF PROJECT/	Project root directory
node_modules/	npm dependencies (not committed to Git)
public/	Static assets served directly
src/	All application source files
assets/	Images, icons, and static media
components/	Feature-specific React components
AddHoliday.jsx	Admin form to add new holidays
AdminPanel.jsx	User management and role control
AuditHistory.jsx	Timestamped audit log viewer
BranchHours.jsx	Branch operational hours manager
ExportModule.jsx	CSV / iCal / PDF export engine
HolidayList.jsx	Main holiday list and calendar view
NotificationCentre.jsx	Compose and manage customer alerts
ServiceDowntime.jsx	Maintenance window scheduler
pages/	Top-level route page components
Home.jsx	Landing page with module grid
Login.jsx	Authentication login page
Signup.jsx	User registration page
styles.css	Global styles for all components
App.jsx	Root router with all route definitions
main.jsx	Application entry point
index.css	Base reset and global typography
package.json	Project metadata and npm scripts
vite.config.js	Vite build configuration

SECTION 10

Functional Requirements

10.1 Authentication Module

ID	Requirement
FR-01	The system shall allow users to register with a unique username, password (min 6 characters), and role selection (Admin/User).
FR-02	The system shall prevent duplicate usernames during registration and display an inline error message.
FR-03	The system shall authenticate users by matching credentials against localStorage records.
FR-04	The system shall redirect unauthenticated users to the Login page when accessing any protected module.
FR-05	The system shall persist login state using localStorage keys: isLoggedIn and username.
FR-06	The system shall allow logged-in users to log out, clearing all session keys from localStorage.

10.2 Holiday Management

ID	Requirement
FR-07	The system shall display all bank holidays in a sortable, filterable list table with columns for name, date, day, type, and state.
FR-08	The system shall provide a month-based calendar grid view as an alternative display mode.
FR-09	The system shall support filtering by Indian state (26 + UTs), holiday type (5 types), month, and free-text search.
FR-10	Admin users shall be able to add holidays via a validated form with name, date, type, region, and description fields.
FR-11	The system shall reject duplicate entries with the same holiday name and date with a specific error message.
FR-12	Admin users shall be able to delete any holiday entry with a browser confirmation dialog.
FR-13	The system shall seed 200+ pre-populated holidays if no localStorage data exists on first load.
FR-14	All holiday additions and deletions shall be written to the audit log with user and timestamp.

10.3 Service Downtime

ID	Requirement
FR-15	Admin users shall be able to schedule maintenance windows specifying system name, date, start/end time, and reason.
FR-16	The system shall support a full status lifecycle: Scheduled → Active → Completed.
FR-17	Admin users shall be able to delete non-completed downtime entries with a confirmation step.
FR-18	All downtime entries shall be automatically written to the audit log on creation.
FR-19	The downtime list shall be filterable by status: All, Scheduled, Active, Completed, Cancelled.

10.4 Branch Hours

ID	Requirement
FR-20	The system shall display operational hours for bank branches across major Indian cities.
FR-21	Admin users shall be able to edit opening time, closing time, and lunch break details per branch.
FR-22	Branch hour changes shall be persisted to localStorage and reflected in the Audit History.

10.5 Notification Centre

ID	Requirement
FR-23	Admin users shall be able to compose and send/save notifications via SMS, Email, or Both channels.
FR-24	The system shall provide three built-in templates: Holiday Closure, System Maintenance, Branch Hours Change.
FR-25	Notifications shall be saveable as drafts with 'Draft' status before final sending.
FR-26	All sent and drafted notifications shall appear in a history list with subject, channel, audience, and status.
FR-27	Users shall be able to preview the full message body of any notification in the history list.

10.6 Audit History

ID	Requirement
FR-28	The system shall log every significant action with: action type (9 types), detail string, acting user, and ISO timestamp.
FR-29	Admin users shall be able to filter the log by action type, user, and free-text keyword search.
FR-30	Admin users shall be able to clear the entire audit log with a browser confirmation dialog.
FR-31	Non-admin users shall receive an 'Admin Access Only' screen and cannot view the audit log.

10.7 Admin Panel

ID	Requirement
FR-32	Admin users shall be able to view all registered users in a searchable, role-filterable table.
FR-33	Admin users shall be able to change any user's role between Admin and User with immediate persistence.
FR-34	Admin users shall be able to delete user accounts with confirmation; self-deletion shall be blocked.
FR-35	Role changes shall be written to the audit log with old role, new role, and acting admin username.

10.8 Export Module

ID	Requirement
FR-36	Users shall be able to download filtered holiday data as a correctly formatted CSV file.
FR-37	Users shall be able to download holiday data as an iCal (.ics) file compatible with Google Calendar, Outlook, and Apple Calendar.
FR-38	Users shall be able to open a print-ready HTML view in a new browser tab for PDF export via browser print.
FR-39	All exports shall respect the active state filter and holiday type filter selections.
FR-40	The export summary shall display the count of holidays selected for export before initiating the download.

FR-31	Non-admin users shall receive an 'Admin Access Only' screen and cannot view the audit log.
-------	--

10.9 Admin Panel

ID	Requirement
FR-32	Admin users shall be able to view all registered users in a searchable, role-filterable table.
FR-33	Admin users shall be able to change any user's role between Admin and User with immediate persistence.
FR-34	Admin users shall be able to delete user accounts with confirmation; self-deletion shall be blocked.
FR-35	Role changes shall be written to the audit log with old role, new role, and acting admin username.

10.10 Export Module

ID	Requirement
FR-36	Users shall be able to download filtered holiday data as a correctly formatted CSV file.
FR-37	Users shall be able to download holiday data as an iCal (.ics) file compatible with Google Calendar, Outlook, and Apple Calendar.
FR-38	Users shall be able to open a print-ready HTML view in a new browser tab for PDF export via browser print.
FR-39	All exports shall respect the active state filter and holiday type filter selections.
FR-40	The export summary shall display the count of holidays selected for export before initiating the download.

SECTION 11

Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	Page load time shall be under 2 seconds on a standard broadband connection. Vite's code-splitting ensures only required module code is loaded.
NFR-02	Usability	All UI interactions shall provide visual feedback within 200ms: loading states on export buttons, inline alert messages for errors and success.
NFR-03	Responsiveness	The portal shall be fully functional on screen widths from 375px (mobile) to 1920px (wide desktop) using CSS Grid and Flexbox.
NFR-04	Security	Role-based access controls shall be enforced on every protected route and action. Admin-only routes check role on render.
NFR-05	Reliability	LocalStorage read/write operations are wrapped in conditional checks to prevent data loss from corrupted or missing keys.
NFR-06	Maintainability	Components are modular with no cross-component shared state outside localStorage. Each module is self-contained.
NFR-07	Accessibility	Interactive elements include descriptive labels, keyboard navigation support, and ARIA-compatible markup.
NFR-08	Consistency	All pages share a consistent header, footer, typography, color palette, and CSS variable system.
NFR-09	Data Integrity	Duplicate holiday entries (same name + date) are rejected at submission time with a specific error message.
NFR-10	Scalability	Architecture supports addition of new modules without restructuring existing ones — new routes simply added to App.jsx.

SECTION 12

Module: Home & Authentication

12.1 Home Page (Home.jsx)

The Home page serves as the central dashboard for the NationalBank Portal. It renders the hero section, the module navigation grid, and the feature highlights section. All module cards check authentication status before navigating — unauthenticated users are redirected to Login.

Component	Description
HeroSection	Full-width banner with portal branding, tagline, and call-to-action
Modules Grid	Featured Holiday List card + two rows of module cards (4 + 3 layout) with navigation guards
Highlights Section	6-card grid showcasing portal capabilities with icons
Header/Footer	Shared navigation bar with dynamic login/logout state

12.2 Login Page (Login.jsx)

The Login page implements localStorage-based authentication. Users enter their username and password; the system searches the 'users' array in localStorage for a matching record. On success, 'isLoggedIn' is set to 'true' and 'username' is stored. Validation errors display inline.

Key Logic: `const validUser = users.find(u => u.username === username && u.password === password)`

On match: `localStorage.setItem('isLoggedIn', 'true') + navigate('/')`

On failure: `setError('Invalid username or password. Please try again.')`

12.3 Signup Page (Signup.jsx)

The Signup page handles new user registration with full validation. It checks for empty fields, minimum password length (6 characters), password confirmation match, and username uniqueness. Users select their role (Admin or User) at registration. On success, the user is redirected to Login after 1.8 seconds with a success message.

- All fields required (username, password, confirm password)
- Password minimum length: 6 characters
- Password and confirm password must match
- Username must be unique across all registered users
- Role selection: User (Standard) or Admin (Full Access)

SECTION 13

Module: Holiday List & Add Holiday

13.1 Holiday List (HolidayList.jsx)

The Holiday List is the core feature of the portal. It displays 200+ bank holidays with rich filtering and two view modes. The component seeds default data from a comprehensive pre-built array if no localStorage data exists, covering all 26 states and 5 Union Territories of India

Feature	Description
Dual View Modes	Toggle between List Table view and Month-wise Calendar Grid view
State Filter	Filter by all 26 Indian states + 5 Union Territories (31 total options)
Type Filter	5 holiday types: National Holiday, Bank Holiday, Restricted Holiday, Regional Holiday, State Holiday
Month Filter	Filter by specific month (January through December)
Search	Free-text search across holiday name and description fields
Admin Actions	Delete button visible only to Admin role users, with confirmation dialog
Seed Data	200+ holidays pre-loaded for 2026 across all states if localStorage is empty
Audit Logging	Every holiday deletion is logged to auditLog with user and timestamp

13.2 Calendar View Logic

The calendar grid renders a 7-column grid (Sun–Sat) for the selected month. Each day cell checks the filtered holiday array for matching dates and displays the holiday name as a colored badge. The month offset is calculated using JavaScript's Date API to correctly position the first day of each month.

13.3 Add Holiday (AddHoliday.jsx)

The Add Holiday page is protected by both authentication and role guards — only Admin users can access it. The form captures holiday name, date, type, region, and optional description. On submission, it checks for duplicates (same name + date) and writes to both 'holidays_v2' and 'holidays' localStorage keys for compatibility. A storage event is dispatched to notify other components.

Dual Key Write: The Add Holiday form writes to two localStorage keys:

- **holidays_v2:** Primary key used by HolidayList component
- **holidays:** Legacy key maintained for Export Module compatibility
- **window.dispatchEvent(new Event('storage')):** Notifies HolidayList to refresh

Field	Details
Holiday Name *	Text input, required — free text, e.g. 'Independence Day'
Date *	Date picker, required — stored as YYYY-MM-DD ISO string
Holiday Type	Select: National, Regional, Bank, Restricted Holiday
Region	Select: National, North, South, East, West, Central
Description	Textarea, optional — brief description of the holiday

SECTION 14

Module: Service Downtime Scheduler

The Service Downtime Scheduler allows bank operations administrators to plan, track, and manage maintenance windows for all critical banking systems. The module implements a full lifecycle management pattern with four distinct statuses.

14.1 Supported Systems

The scheduler supports 10 distinct banking systems:

- Core Banking System
 - Internet Banking
 - Mobile Banking App
 - ATM Network
 - NEFT / RTGS
- UPI Services
 - Card Services
 - Customer Portal
 - Loan Management System
 - Trade Finance Module

14.2 Status Lifecycle

Status	Color	Meaning	Admin Actions Available
Scheduled	Blue	Maintenance window is planned and upcoming	Mark as Active, Delete
Active	Amber	Maintenance is currently in progress	Mark as Completed
Completed	Green	Maintenance window has finished successfully	None (read-only)
Cancelled	Red	Maintenance window was cancelled before start	Delete

14.3 Form Fields & Validation

Field	Details
Affected System	Dropdown with 10 banking system options
Date	Date picker for the maintenance date
Start Time	Time picker (HH:MM format)
End Time	Time picker (HH:MM format)
Reason / Notes	Textarea — description of maintenance purpose and scope

All fields are required. On submission, a new entry is created with status 'Scheduled', the current user's name, and an ISO timestamp. The entry is prepended to the downtimes array (newest first) and the audit log is updated.

SECTION 15

Module: Branch Hours Management

The Branch Hours module enables bank administrators to maintain accurate operational hours for all major bank branches across India. All logged-in users can view branch hours; only admins can edit them. The module supports inline editing with immediate localStorage persistence.

15.1 Branch Data Structure

Each branch entry contains:

- **name**: Branch/city name (e.g., 'Head Office — Hyderabad')
- **weekdays**: Opening and closing time for Monday–Friday
- **saturday**: Opening and closing time (typically half-day)
- **sunday**: Status ('Closed')
- **lunch**: Lunch break window (e.g., '1:00 PM – 2:00 PM')
- **holidays**: Notes on special closure policies

15.2 Cities Covered

- Hyderabad (Head Office)
- Mumbai
- Delhi
- Chennai
- Bengaluru
- Kolkata
- Ahmedabad
- Pune

15.3 Edit Workflow

Step 1: Admin clicks the 'Edit' button on any branch row — inline edit mode activates for that branch only.

Step 2: Time fields (weekday open/close, Saturday open/close, lunch break) become editable inputs.

Step 3: Admin modifies the required fields and clicks 'Save' — changes persist to localStorage under 'branchHours'.

Step 4: The edit is logged to the audit trail as 'Updated Branch Hours' with the branch name as detail.

Step 5: Clicking 'Cancel' reverts all changes to the last saved state without writing to localStorage.

SECTION 16

Module: Notification Centre

The Notification Centre allows administrators to compose and manage customer alert messages. It supports three delivery channels, five audience segments, three built-in message templates, and a full draft-to-sent workflow with complete notification history.

16.1 Delivery Channels

Channel	Color Code	Description
SMS	Green	Short message to customer's registered mobile number
Email	Blue	Formatted email to customer's registered email address
Both	Purple	Simultaneous SMS and email delivery to all registered contacts

16.2 Audience Segments

Five audience segments are available for targeted communication:

- All Customers
- Business Accounts
- Retail Customers
- Loan Customers
- Priority Banking

16.3 Built-in Templates

Template: Holiday Closure

Pre-filled subject and body for branch closure announcements due to holidays. Includes placeholder text for holiday name and date.

Template: System Maintenance

Template for scheduled IT maintenance notifications. Includes system name, date, and start/end time placeholders.

Template: Branch Hours Change

Template for notifying customers of revised operational hours. Includes branch name, effective date, and new hours placeholders.

SECTION 17

Module: Audit History

The Audit History module provides a comprehensive, tamper-visible record of all significant actions performed across the portal. It is restricted to Admin users only and serves as the primary governance and compliance tool for operations management.

17.1 Tracked Action Types

Action Type	Indicator	Trigger
Added Holiday	Green	A new holiday was added via the Add Holiday form
Deleted Holiday	Red	An existing holiday was removed from the calendar
Added Downtime	Amber	A new maintenance window was scheduled
Updated Branch Hours	Blue	Branch operational hours were modified and saved
Sent Notification	Purple	A customer notification was composed and sent
Saved Notification Draft	Grey	A notification was saved as a draft for later
User Login	Navy	A user authenticated successfully into the portal
User Registered	Green	A new user account was created via Signup
Role Changed	Amber	An admin modified a user's role assignment

17.2 Audit Entry Schema

Each audit log entry contains:

- **id**: Unique identifier (Date.now()) timestamp
- **action**: One of the 9 action type strings
- **detail**: Human-readable description (e.g., 'Republic Day on 2026-01-26')
- **user**: Username of the person who performed the action
- **at**: ISO 8601 timestamp (e.g., '2026-01-01T08:00:00.000Z')

SECTION 18

Module: Admin Panel

The Admin Panel provides full user management capabilities to administrators. It is protected by both authentication and role checks — non-admin users see an 'Admin Access Only' screen. The panel provides a searchable, filterable view of all registered users with inline role editing.

18.1 Features

Feature	Description
User Table	Displays all registered users with: username, role badge, active status, and action buttons
Search	Real-time search by username — filters the table as the admin types
Role Filter	Filter users by role: All, Admin, User — with counts in stat buttons
Edit Role	Click 'Edit Role' to toggle a dropdown for that user's row, then 'Save' to apply
Delete User	Remove a user account with a browser confirmation dialog
Self-Protection	The logged-in admin's own row has no delete or edit buttons — prevents accidental self-lockout
Audit Logging	Every role change is logged: 'username → newRole' with timestamp and acting admin

18.2 Role Change Flow

- Step 1:** Admin finds the target user by searching or scrolling the user table.
- Step 2:** Admin clicks 'Edit Role' — the role column for that row becomes a select dropdown.
- Step 3:** Admin selects the new role (Admin or User) and clicks '✓ Save'.
- Step 4:** The users array in localStorage is updated with the new role mapping.
- Step 5:** A success alert displays: 'Role updated for [username] → [newRole]'.
- Step 6:** The audit log receives a new entry: action='Role Changed', detail='[username] → [newRole]'.

SECTION 19

Module: Export Module

The Export Module allows users to download holiday calendar data in three industry-standard formats. Export results respect all active filter selections (state and holiday type), enabling targeted exports for specific regions or holiday categories.

19.1 Export Formats

Format	Extension	Badge	Use Case	Implementation
CSV Spreadsheet	.csv	Universal	Open in Excel, Google Sheets, or any spreadsheet app	Generates comma-separated text with headers; triggers Blob download via anchor element
PDF / Print	Browser Print	Print-Ready	Save as PDF using browser's Print to PDF option	Opens a new tab with a styled HTML table; calls window.print() after 500ms delay
iCal Calendar	.ics	Calendar Sync	Import into Google Calendar, Apple Calendar, Outlook	Generates RFC 5545-compliant VCALENDAR with VEVENT blocks; triggers .ics Blob download

19.2 CSV Export Structure

The CSV file contains the following columns:

- # (row number)
- Holiday Name
- Date (DD Mon YYYY)
- Day (full weekday name)
- Type
- State / Scope
- Description

19.3 iCal Export Structure

iCal File Properties:

- PRODID: -//NationalBank//Holiday Calendar 2026//EN
- CALSCALE: GREGORIAN · METHOD: PUBLISH
- X-WR-TIMEZONE: Asia/Kolkata (IST)
- Eachholiday→oneVEVENTwithDTSTART(all-day),SUMMARY,DESCRIPTION, CATEGORIES
- Specialcharactersinname/descriptionareescapedperRFC5545(commas,semicolons, newlines)

19.4 Filter Interaction

The export summary dynamically updates to show the count of holidays matching the active filters. The download button is disabled when zero holidays match the filters, preventing empty exports. Filters are applied client-side from the localStorage 'holidays' array.

SECTION 20

Version Roadmap

The NationalBank Portal was developed iteratively across three versions, each adding functional depth and complexity. This incremental approach follows agile development principles and aligns with the FEDF curriculum's module progression.

Version	Status	Modules Delivered	Key Features
V1 — Foundation	Done	Home, Login, Signup	Hero page with module grid, localStorage authentication, role-based signup, navigation guards, responsive CSS system, shared Header/Footer components
V2 — Calendar & Holidays	Done	HolidayList, AddHoliday, BranchHours	200+ holiday seed data for 26 states, dual list/calendar view, 4-way filtering (state, type, month, search), admin-only Add Holiday form with duplicate detection, branch hours display
Final — Full Operations	Done	ServiceDowntime, NotificationCentre, AuditHistory, AdminPanel, ExportModule	Downtime lifecycle management, notification templates and draft workflow, 9-action audit log, user role management, CSV/iCal/PDF export with filter support, complete localStorage persistence

20.1 Development Timeline

Milestone	Deliverables
V1 Kickoff	Project scaffolding with Vite, global CSS system, color palette definition
V1 Complete	Home, Login, Signup fully functional with localStorage auth and route guards
V2 Kickoff	HolidayList with seed data, BranchHours display
V2 Complete	AddHoliday admin form, calendar grid view, full filtering
Final Kickoff	ServiceDowntime, NotificationCentre, AuditHistory, AdminPanel, ExportModule
Final Complete	All 10 modules functional, full localStorage persistence, responsive QA, documentation

SECTION 21

Acceptance Criteria

The following acceptance criteria define the minimum conditions that must be satisfied for each module to be considered complete and acceptable for submission.

ID	Module	Criterion
AC-01	Auth	A new user can register with unique username, password, and role; redirected to login on success
AC-02	Auth	An existing user can log in and is redirected to the Home page
AC-03	Auth	An unauthenticated user clicking any module is redirected to the Login page
AC-04	Auth	A logged-in user can log out; subsequent protected route access is blocked
AC-05	Holiday	The Holiday List displays all seeded holidays on load
AC-06	Holiday	Applying a state filter updates the list within 300ms
AC-07	Holiday	The calendar grid shows holidays on their correct dates
AC-08	Holiday	Admin sees a delete button; non-admin does not
AC-09	Add Holiday	Submitting with empty required fields shows an inline error message
AC-10	Add Holiday	Submitting a duplicate (same name + date) is rejected with a specific error
AC-11	Add Holiday	A valid holiday is added and user redirected after 1.8 seconds
AC-12	Add Holiday	Page is inaccessible to non-admin users who see 'Access Restricted'
AC-13	Downtime	Admin can schedule a downtime; it appears with 'Scheduled' status
AC-14	Downtime	Admin can advance status: Scheduled → Active → Completed
AC-15	Downtime	Each new downtime entry appears in Audit History
AC-16	Branch	All branches visible with correct hours to all logged-in users
AC-17	Branch	Admin can edit branch hours; change persists on page reload
AC-18	Branch	Branch hour edits are reflected in Audit History
AC-19	Notification	Admin can compose using a template; subject and body pre-filled
AC-20	Notification	'Send Now' saves notification as 'Sent' and logs to audit
AC-21	Notification	'Save Draft' saves notification with 'Draft' status

AC-22	Notification	All notifications appear in history list with preview capability
AC-23	Audit	All tracked actions appear in audit log with correct user and timestamp
AC-24	Audit	Filtering by action type and user narrows results correctly
AC-25	Audit	Non-admin users receive 'Admin Access Only' screen
AC-26	Admin	Admin sees all registered users with their roles
AC-27	Admin	Admin can change a user's role; change saved immediately
AC-28	Admin	Admin cannot delete their own account; delete button hidden for self
AC-29	Export	CSV download triggers correctly formatted file download
AC-30	Export	iCal download creates .ics file importable into Google Calendar
AC-31	Export	PDF/Print opens a print-ready browser window
AC-32	Export	Export respects active state and type filters
AC-33	General	Portal renders correctly on mobile (375px) without horizontal scrolling
AC-34	General	All pages include the consistent header with portal branding and navigation
AC-35	General	Success and error alerts display inline and auto-clear

SECTION 22

Assumptions & Constraints

22.1 Technical Assumptions

- All data persistence is handled via browser localStorage; no backend server or database is required.
- The portal targets modern browsers (Chrome 100+, Firefox 100+, Edge 100+, Safari 15+) supporting ES2022+, localStorage, and CSS Grid.
- Internet access is not required after initial page load; the application functions entirely client-side.
- The project is built with Vite as the build tool and is deployable as a static site (e.g., Netlify, GitHub Pages).
- React Router's HashRouter is used (instead of BrowserRouter) to ensure correct routing on static file hosts without server configuration.
- User passwords are stored as plain text in localStorage — acceptable for an academic front-end project but not for production use.

22.2 Business Assumptions

- Notification delivery (SMS/email) is simulated — messages are logged locally and not actually transmitted to any external service.
- Role assignment during signup is trust-based; a production system would require admin approval for elevated roles.
- The holiday calendar covers 2026 by default, consistent with the project's current academic year.
- All date and time values are stored and displayed in IST (Indian Standard Time, UTC+5:30).
- Initial seed data for 200+ holidays, downtime windows, and branch hours is pre-populated to demonstrate functionality.
- The 200+ seeded holidays are representative; they do not constitute official RBI or government holiday notifications.

22.3 Known Constraints & Limitations

Constraint	Details
Storage Limit	Browser localStorage has a ~5MB limit per origin. With 200+ holidays and full audit logs, this is sufficient for demo purposes but would require a backend database for production scale.
No Real Notifications	SMS and email notifications are simulated. A production implementation would integrate Twilio (SMS), SendGrid (email), or similar APIs.
Single Browser Session	Data is per-browser and per-origin. Multiple users on different browsers cannot share the same data without a backend.
No Password Hashing	Passwords are stored as plain text. A production system requires bcrypt or similar hashing before any deployment.
No Token-Based Auth	Authentication relies on localStorage flags. A production system requires JWT or session tokens with server-side validation.
Static Holiday Data	Holiday data for 2026 is pre-seeded. Annual updates require code-level changes rather than an admin interface for bulk imports.

SECTION 23

Conclusion

The Bank Holiday & Service Calendar — NationalBank Portal — represents the successful delivery of a complete, production-quality React.js web application encompassing 10 functional modules, role-based access control, real-time audit logging, and multi-format data export. The project demonstrates mastery of the complete FEDF curriculum from basic component rendering to advanced state management and client-side routing.

Project Achievements Summary

Achievement	Description
10 Modules Delivered	All required modules implemented and functional: Home, Login, Signup, HolidayList, AddHoliday, ServiceDowntime, BranchHours, NotificationCentre, AuditHistory, AdminPanel, ExportModule.
200+ Holiday Entries	Comprehensive 2026 holiday data seeded across all 26 Indian states and 5 Union Territories, covering National, Bank, Regional, and Restricted holiday types.
Full RBAC Implementation	Role-based access control enforced at every route and action level — Admin and User roles with complete permission separation.
3 Export Formats	Holiday data exportable as CSV (for spreadsheets), iCal/ICS (for calendar apps), and Print-to-PDF — all respecting active filters.
Complete Audit Trail	9 distinct action types logged with user identity and ISO timestamps — providing full governance visibility for all portal changes.
Responsive UI	Functional across 375px mobile to 1920px desktop using CSS Grid and Flexbox with consistent design language throughout.
React Best Practices	Functional components, useState/useEffect hooks, controlled forms, unidirectional data flow, and component composition applied throughout.

Learning Outcomes

This project provided hands-on experience with the complete React.js development lifecycle — from initial project scaffolding with Vite to final deployment as a static site. Key learning outcomes include component-based architecture design, controlled form patterns, client-side routing with route protection, localStorage as a lightweight data layer, CSS design systems with variables, and iterative feature delivery across multiple versions.

Future Scope

- Backend API integration using Node.js/Express with MongoDB or PostgreSQL for true multi-user data sharing.
- Real SMS/email delivery via Twilio and SendGrid APIs, replacing the current simulation layer.
- JWT-based authentication with refresh tokens, replacing localStorage session flags.
- RBI holiday API integration for automatic annual holiday data updates.
- Push notification support for real-time browser alerts on new holiday additions.
- Multi-language support (Hindi, Telugu, Tamil, etc.) using i18n localization libraries.
- Bulk holiday import via CSV file upload for operations teams managing large data sets.

G. Hansika Reddy (2520030237) K. Anusri (2520030432)

Department of Front End Development Frameworks and UI Engineering · Batch No. 20

Under the Guidance of Dr. Lakshmi Prasanna · 2026